

UJIAN TENGAH SEMESTER ANTARA
PEMROGRAMAN MOBILE LANJUT



Disusun oleh:

Rama Pramudya Wibisana

2022320019

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS INFORMATIKA

UNIVERSITAS BINA INSANI

BEKASI

2024

SOAL TEORI

1. Langkah-langkah menginstall Flutter SDK pada Windows:

- Download Flutter SDK di website resmi Flutter dan unduh file ZIP Flutter SDK untuk Windows.
- Ekstrak File ZIP ke directory yang diinginkan, misalnya C:\flutter.
- Tambahkan Flutter ke PATH:
 - Buka Control Panel > System and Security > System > Advanced system settings > Environment Variables.
 - Tambahkan path ke direktori flutter\bin ke variabel PATH. Misalnya, tambahkan C:\flutter\bin.
- Buka Command Prompt atau PowerShell dan jalankan flutter doctor untuk memeriksa apakah ada dependensi yang perlu diinstal.
- Download dan install Android Studio dan tambahkan Android SDK.
- Buka Android Studio dan install plugin Flutter dan Dart.
- Install device emulator atau hubungkan hard device.

2. Struktur Proyek Flutter

Secara umum, struktur proyek Flutter terdiri dari beberapa folder dan file yang masing-masing memiliki fungsi khusus. Berikut adalah penjelasan tentang struktur utama proyek Flutter:

1. Folder `android`:

- Berisi kode dan konfigurasi yang diperlukan untuk membangun aplikasi Android.

2. Folder `ios`:

- Berisi kode dan konfigurasi yang diperlukan untuk membangun aplikasi iOS.

3. Folder `lib`:

- Tempat utama untuk menulis kode Dart Anda. Biasanya berisi file `main.dart` dan file lain yang mendefinisikan widget dan logika aplikasi Anda.

4. Folder ``test``:

- Berisi file untuk menulis tes unit dan tes widget aplikasi Anda.

5. File ``pubspec.yaml``:

- File konfigurasi yang digunakan untuk mengelola dependensi, metadata proyek, dan konfigurasi build. Di sini, Anda dapat menambahkan dependensi package, font, aset, dan pengaturan lainnya.

6. File ``pubspec.lock``:

- Berisi informasi tentang versi package yang tepat yang digunakan oleh proyek Anda. File ini dihasilkan otomatis dan tidak perlu diubah secara manual.

7. File ``README.md``:

- File dokumentasi untuk proyek Anda. Biasanya berisi deskripsi proyek, cara menginstal dan menjalankannya, dan informasi lain yang relevan.

8. File ``analysis_options.yaml`` (opsional):

- Digunakan untuk mengonfigurasi analisis statis kode Dart, seperti aturan linter dan format kode.

Fungsi dari Beberapa File Utama

1. ``pubspec.yaml``:

- Fungsi: Menyimpan metadata proyek dan daftar dependensi. Di sini Anda dapat menentukan paket-paket eksternal yang digunakan dalam proyek, seperti ``provider``, ``http``, dll., serta aset seperti gambar dan font.

- Contoh:

```
``yaml
name: my_flutter_app
description: A new Flutter project.
```

dependencies:

flutter:

 sdk: flutter

 cupertino_icons: ^1.0.2

flutter:

 assets:

 - assets/images/

 ...

2. `main.dart`:

- Fungsi: Titik masuk utama aplikasi Flutter. File ini berisi fungsi `main()` yang menjalankan aplikasi dan memulai widget utama. Biasanya, di sini Anda menentukan widget root yang menampilkan antarmuka pengguna aplikasi.

- Contoh:

```
``dart
```

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Flutter Demo',  
      theme: ThemeData(  
        primarySwatch: Colors.blue,  
      ),  
      home: MyHomePage(),  
    );  
  }
```

```

    }
  }

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Flutter Demo Home Page'),
      ),
      body: Center(
        child: Text('Hello, World!'),
      ),
    );
  }
}

```

3. Folder `lib`:

- Fungsi: Tempat utama untuk menulis kode Dart Anda. Biasanya berisi file `main.dart` dan file-file lain yang mengatur logika dan antarmuka pengguna aplikasi Anda. Anda dapat membuat subfolder di dalam `lib` untuk mengorganisir kode dengan lebih baik, misalnya folder `models`, `views`, `widgets`, dll.

Contoh Sederhana Proyek Flutter

Berikut adalah contoh sederhana proyek Flutter dengan minimal satu widget:

1. Struktur Proyek:

```
...  
  
my_flutter_app/  
├── android/  
├── ios/  
├── lib/  
│   ├── main.dart  
├── test/  
├── pubspec.yaml  
└── README.md  
...
```

2. File `lib/main.dart`:

```
``dart  
  
import 'package:flutter/material.dart';  
  
void main() {  
  runApp(MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Simple App',  
      theme: ThemeData(  
        primarySwatch: Colors.blue,  
      ),  
    );  
  }  
}
```

```

home: Scaffold(
  appBar: AppBar(
    title: Text('Simple App'),
  ),
  body: Center(
    child: Text('Hello, Flutter!'),
  ),
),
);
}
}
...

```

- Deskripsi:

- `MyApp` adalah widget root yang menjalankan aplikasi dengan `MaterialApp`.
- `Scaffold` adalah widget yang menyediakan struktur dasar antarmuka pengguna dengan `AppBar` dan `body`.
- `Text` adalah widget yang menampilkan teks "Hello, Flutter!" di tengah layar.

3. Berikut adalah lima widget dasar Flutter yang sering digunakan dan contoh penggunaannya dalam membangun tampilan sederhana, seperti form login:

1. `Container`

Penjelasan:

- Widget `Container` digunakan untuk membungkus widget lain dan memberikan padding, margin, ukuran, warna latar belakang, dan gaya border. Ini adalah widget yang fleksibel dan sering digunakan untuk membuat layout yang terstruktur.

Contoh Penggunaan:

```
``dart
Container(
  padding: EdgeInsets.all(16.0),
  margin: EdgeInsets.symmetric(vertical: 10.0),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(8.0),
    boxShadow: [
      BoxShadow(
        color: Colors.grey.withOpacity(0.2),
        spreadRadius: 2,
        blurRadius: 5,
      ),
    ],
  ),
  child: Text('Hello, Flutter!'),
)
```

2. `TextField`

Penjelasan:

- Widget `TextField` digunakan untuk menerima input teks dari pengguna. Ini adalah widget penting dalam form untuk menangani input seperti nama pengguna dan kata sandi.

Contoh Penggunaan:

```
``dart
TextField(
  decoration: InputDecoration(
    labelText: 'Username',
```



```

border: OutlineInputBorder(),
prefixIcon: Icon(Icons.person),
),
)
...

```

3. `RaisedButton` (atau `ElevatedButton`)

Penjelasan:

- Widget `RaisedButton` (sekarang direkomendasikan menggunakan `ElevatedButton`) digunakan untuk membuat tombol yang memiliki elevasi dan bisa ditekan. Ini sering digunakan untuk aksi seperti mengirim form.

Contoh Penggunaan:

```

```dart
ElevatedButton(
 onPressed: () {
 // Aksi saat tombol ditekan
 print('Login button pressed');
 },
 child: Text('Login'),
 style: ElevatedButton.styleFrom(
 primary: Colors.blue, // Background color
 onPrimary: Colors.white, // Text color
),
)
...

```

#### 4. `Column`

Penjelasan:

- Widget `Column` digunakan untuk menyusun widget secara vertikal. Ini sangat berguna untuk menyusun beberapa widget di dalam layout secara berurutan.

Contoh Penggunaan:

```
``dart
Column(
 crossAxisAlignment: CrossAxisAlignment.start,
 children: <Widget>[
 Text('Username'),
 TextField(
 decoration: InputDecoration(
 border: OutlineInputBorder(),
),
),
 SizedBox(height: 16.0),
 Text('Password'),
 TextField(
 obscureText: true,
 decoration: InputDecoration(
 border: OutlineInputBorder(),
),
),
 SizedBox(height: 24.0),
 ElevatedButton(
 onPressed: () {
 // Aksi saat tombol ditekan
 print('Login button pressed');
 },
 child: Text('Login'),
),
],
)
```

```
),
],
)
```
```

5. `Scaffold`

Penjelasan:

- Widget `Scaffold` menyediakan struktur dasar untuk tata letak aplikasi, termasuk app bar, drawer, bottom navigation bar, dan body. Ini adalah widget dasar yang membentuk kerangka umum aplikasi.

Contoh Penggunaan:

```
```dart  
Scaffold(
 appBar: AppBar(
 title: Text('Login Form'),
),
 body: Padding(
 padding: EdgeInsets.all(16.0),
 child: Column(
 crossAxisAlignment: CrossAxisAlignment.start,
 children: <Widget>[
 Text('Username'),
 TextField(
 decoration: InputDecoration(
 border: OutlineInputBorder(),
),
),
 SizedBox(height: 16.0),
 Text('Password'),
 TextField(
 decoration: InputDecoration(
 border: OutlineInputBorder(),
),
),
],
),
),
)
```

```

 obscureText: true,
 decoration: InputDecoration(
 border: OutlineInputBorder(),
),
),
 SizedBox(height: 24.0),
 ElevatedButton(
 onPressed: () {
 // Aksi saat tombol ditekan
 print('Login button pressed');
 },
 child: Text('Login'),
),
],
),
)
```

```

Form Login Sederhana

Gabungan dari widget-widget di atas dalam sebuah form login sederhana:

```

```dart
import 'package:flutter/material.dart';

void main() {
 runApp(MyApp());
}

class MyApp extends StatelessWidget {
 @override

```

```
Widget build(BuildContext context) {
 return MaterialApp(
 title: 'Login Form',
 theme: ThemeData(
 primarySwatch: Colors.blue,
),
 home: LoginScreen(),
);
}
```

```
class LoginScreen extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Login Form'),
),
 body: Padding(
 padding: EdgeInsets.all(16.0),
 child: Column(
 crossAxisAlignment: CrossAxisAlignment.start,
 children: <Widget>[
 Text('Username'),
 SizedBox(height: 8.0),
 TextField(
 decoration: InputDecoration(
 border: OutlineInputBorder(),
),
),
 SizedBox(height: 16.0),
 Text('Password'),
```

```

 SizedBox(height: 8.0),
 TextField(
 obscureText: true,
 decoration: InputDecoration(
 border: OutlineInputBorder(),
),
),
 SizedBox(height: 24.0),
 ElevatedButton(
 onPressed: () {
 // Aksi saat tombol ditekan
 print('Login button pressed');
 },
 child: Text('Login'),
),
],
),
);
}
}
```

```

Dengan menggunakan widget-widget ini, maka kita dapat membuat tampilan form login sederhana dan menambahkan lebih banyak fungsionalitas seiring berjalannya waktu.

4. Dalam Flutter, konsep layout berhubungan dengan cara widget diatur di layar dan bagaimana mereka menyesuaikan diri terhadap ukuran dan posisi relatif satu sama lain. Flutter menggunakan sistem layout berbasis widget di mana hampir semua aspek tata letak dikelola oleh widget.

Konsep Layout dalam Flutter

1. Parent-Child Relationship:

- Widget di Flutter sering kali memiliki struktur parent-child di mana satu widget dapat mengandung beberapa widget lainnya. Parent widget bertanggung jawab untuk menentukan tata letak dan posisi dari child widget-nya.

2. Constraints:

- Widget di Flutter bekerja dengan sistem constraint. Parent widget memberikan batasan ukuran (constraints) kepada child widget, dan child widget harus mematuhi batasan ini saat melakukan layout.

3. Flexibility:

- Beberapa widget menyediakan fleksibilitas dalam mengatur ukuran dan posisi child widget. Misalnya, `Row` dan `Column` menyediakan cara untuk menyusun widget secara horizontal dan vertikal.

Perbedaan antara `Row`, `Column`, dan `Stack`

1. `Row`:

- Deskripsi: Widget `Row` menyusun child widget-nya secara horizontal (dari kiri ke kanan). Ini berguna ketika Anda ingin menampilkan beberapa elemen dalam satu baris.

- Kelebihan: Mudah untuk membuat layout horizontal.

- Contoh:

```
```dart
```

```
Row(
 children: <Widget>[
 Icon(Icons.star),
 Text('Star'),
 Icon(Icons.star),
],
)
```

```
)
...
```

- Pengaturan Tata Letak: `Row` menempatkan child widget-nya dari kiri ke kanan, dengan opsi untuk mengatur alignment, spacing, dan ukuran relatif menggunakan properti seperti `mainAxisAlignment`, `crossAxisAlignment`, dan `mainAxisSize`.

## 2. `Column`:

- Deskripsi: Widget `Column` menyusun child widget-nya secara vertikal (dari atas ke bawah). Ini berguna ketika Anda ingin menampilkan beberapa elemen dalam satu kolom.

- Kelebihan: Mudah untuk membuat layout vertikal.

- Contoh:

```
```dart  
Column(  
  crossAxisAlignment: CrossAxisAlignment.start,  
  children: <Widget>[  
    Text('Item 1'),  
    Text('Item 2'),  
    Text('Item 3'),  
  ],  
)  
...
```

- Pengaturan Tata Letak: `Column` menempatkan child widget-nya dari atas ke bawah, dengan opsi untuk mengatur alignment, spacing, dan ukuran relatif menggunakan properti seperti `mainAxisAlignment`, `crossAxisAlignment`, dan `mainAxisSize`.

3. `Stack`:

- Deskripsi: Widget `Stack` memungkinkan Anda menempatkan child widget di atas satu sama lain, dengan widget yang ditambahkan lebih terakhir muncul di atas widget yang ditambahkan sebelumnya. Ini berguna untuk membuat desain yang lebih kompleks di mana elemen-elemen saling bertumpuk.

- Kelebihan: Fleksibilitas untuk menempatkan widget di atas satu sama lain.
- Contoh:

```

dart
Stack(
  children: <Widget>[
    Container(
      color: Colors.blue,
      width: 100,
      height: 100,
    ),
    Positioned(
      top: 20,
      left: 20,
      child: Container(
        color: Colors.red,
        width: 50,
        height: 50,
      ),
    ),
  ],
)

```

- Pengaturan Tata Letak: `Stack` menempatkan child widget-nya di atas satu sama lain. Anda dapat menggunakan widget `Positioned` untuk menentukan posisi relatif dari child widget dalam `Stack`.

Cara Mengatur Tata Letak Anak-Anak Widget di dalam Parent Widget

1. Menggunakan Properti Alignment:

- Widget seperti `Row` dan `Column` memiliki properti seperti `mainAxisAlignment` dan `crossAxisAlignment` untuk mengatur tata letak child widget secara horizontal dan vertikal.

2. Menggunakan Padding dan Margin:

- Widget ``Padding`` dapat digunakan untuk memberikan ruang di sekitar child widget, sedangkan ``Container`` dapat digunakan dengan properti ``margin`` untuk memberikan ruang di luar batas widget.

3. Menggunakan Flex:

- Di dalam ``Row`` dan ``Column``, Anda dapat menggunakan widget ``Expanded`` atau ``Flexible`` untuk mengatur seberapa banyak ruang yang harus diambil oleh masing-masing child widget.

4. Menggunakan Layout Widgets Lainnya:

- Widget seperti ``Align``, ``Center``, dan ``FractionallySizedBox`` dapat digunakan untuk mengatur posisi dan ukuran child widget dengan lebih presisi.

Dengan memahami cara kerja ``Row``, ``Column``, dan ``Stack``, serta cara mengatur tata letak menggunakan berbagai properti dan widget tambahan, kita dapat membuat tampilan yang responsif dan terorganisir dengan baik di aplikasi Flutter.

5. Navigasi antar halaman dalam aplikasi Flutter dilakukan menggunakan ``Navigator``. ``Navigator`` adalah widget yang mengelola stack halaman (screen) dan memungkinkan Anda untuk berpindah dari satu halaman ke halaman lainnya. Ada beberapa metode untuk melakukan navigasi, termasuk ``push``, ``pop``, dan ``pushReplacement``.

Navigasi Antar Halaman Menggunakan ``Navigator``

1. ``Navigator.push``:

- Digunakan untuk menambahkan halaman baru ke stack navigasi. Halaman baru ini akan ditampilkan di atas halaman saat ini.

2. ``Navigator.pop``:

- Digunakan untuk menghapus halaman saat ini dari stack navigasi dan kembali ke halaman sebelumnya.

3. `Navigator.pushReplacement``:

- Digunakan untuk menggantikan halaman saat ini dengan halaman baru. Halaman baru menggantikan halaman yang sedang ditampilkan.

Passing Data Antar Halaman

Untuk mengirim data dari satu halaman ke halaman lainnya, Anda dapat melakukannya melalui parameter constructor pada halaman yang baru. Data dapat dikirim saat melakukan navigasi menggunakan metode `Navigator.push``.

Contoh Implementasi Navigasi Sederhana dengan Dua Halaman

Berikut adalah contoh implementasi navigasi sederhana dengan dua halaman dan passing data antar halaman.

1. Halaman Pertama (`home_screen.dart``)

```
``dart
import 'package:flutter/material.dart';
import 'package:your_app/second_screen.dart'; // Pastikan untuk mengimpor halaman
kedua

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
```

```

return MaterialApp(
  title: 'Navigation Example',
  theme: ThemeData(
    primarySwatch: Colors.blue,
  ),
  home: HomeScreen(),
);
}
}

```

```

class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Home Screen'),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            // Navigasi ke halaman kedua dengan data
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => SecondScreen(
                  data: 'Hello from Home Screen!',
                ),
              ),
            );
          },
          child: Text('Go to Second Screen'),
        ),
      ),
    );
  }
}

```

```
    ),  
  );  
}  
}  
...
```

2. Halaman Kedua (`second\_screen.dart`)

```
``dart  
import 'package:flutter/material.dart';  
  
class SecondScreen extends StatelessWidget {  
  final String data;  
  
  // Constructor untuk menerima data dari halaman pertama  
  SecondScreen({required this.data});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Second Screen'),  
      ),  
      body: Center(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: <Widget>[  
            Text('Received Data: $data'),  
            SizedBox(height: 20),  
            ElevatedButton(  
              onPressed: () {  
                // Kembali ke halaman sebelumnya
```

```

        Navigator.pop(context);
      },
      child: Text('Go Back'),
    ),
  ],
),
),
);
}
}
```

```

### **Penjelasan Kode:**

- `HomeScreen`:
  - Halaman utama yang menampilkan tombol. Ketika tombol ditekan, navigasi dilakukan ke `SecondScreen` dengan menggunakan `Navigator.push`.
  - Data yang dikirim adalah string "Hello from Home Screen!".
- `SecondScreen`:
  - Halaman kedua yang menerima data melalui constructor. Data ini ditampilkan di `Text` widget.
  - Tombol pada halaman kedua memungkinkan pengguna untuk kembali ke halaman sebelumnya dengan menggunakan `Navigator.pop`.

### **Cara Kerja:**

1. Saat tombol "Go to Second Screen" ditekan di `HomeScreen`, `Navigator.push` akan menambahkan `SecondScreen` ke stack navigasi dan menampilkan halaman tersebut.

2. Data yang dikirim melalui constructor `SecondScreen` ditampilkan di halaman kedua.

3. Ketika tombol "Go Back" ditekan di `SecondScreen`, `Navigator.pop` akan menghapus `SecondScreen` dari stack dan kembali ke `HomeScreen`.

## SOAL PRAKTEK

### main.dart

```
// ignore_for_file: use_key_in_widget_constructors,
prefer_const_constructors_in_immutables,
library_private_types_in_public_api, prefer_const_constructors

import 'package:flutter/material.dart';

class ShapeDetailPage extends StatefulWidget {
 final String shape;

 ShapeDetailPage({required this.shape});

 @override
 _ShapeDetailPageState createState() => _ShapeDetailPageState();
}

class _ShapeDetailPageState extends State<ShapeDetailPage> {
 final _formKey = GlobalKey<FormState>();
 double resultArea = 0.0;
 double resultPerimeter = 0.0;

 TextEditingController param1Controller = TextEditingController();
 TextEditingController param2Controller = TextEditingController();
 TextEditingController param3Controller = TextEditingController();

 String param1Label = '';
 String param2Label = '';
 String param3Label = '';

 void calculate() {
 double param1 = double.tryParse(param1Controller.text) ?? 0.0;
 double param2 = double.tryParse(param2Controller.text) ?? 0.0;
 double param3 = double.tryParse(param3Controller.text) ?? 0.0;
```

```

 setState(() {
 switch (widget.shape) {
 case 'Kubus':
 resultArea = 6 * param1 * param1; // Luas Kubus
 resultPerimeter = 12 * param1; // Keliling Kubus
 break;
 case 'Bola':
 resultArea = 4 * 3.1416 * param1 * param1; // Luas Bola
 resultPerimeter = 2 * 3.1416 * param1; // Keliling Bola
 break;
 case 'Silinder':
 resultArea = 2 * 3.1416 * param1 * (param1 + param2); // Luas
 resultPerimeter = 2 * 3.1416 * param1; // Keliling Silinder
 break;
 case 'Kerucut':
 resultArea = 3.1416 * param1 * (param1 + param2); // Luas
 resultPerimeter = 3.1416 * param1; // Keliling Kerucut
 break;
 case 'Balok':
 resultArea = 2 * (param1 * param2 + param2 * param3 + param3 *
 param1); // Luas Balok
 resultPerimeter = 4 * (param1 + param2 + param3); // Keliling
 break;
 }
 });
 }

 @override
 void initState() {
 super.initState();
 setLabels();
 }

 void setLabels() {
 switch (widget.shape) {
 case 'Kubus':
 param1Label = 'Panjang Sisi';
 break;
 case 'Bola':
 param1Label = 'Jari-jari';
 break;
 case 'Silinder':
 param1Label = 'Jari-jari Alas';

```



```

 param2Label = 'Tinggi';
 break;
 case 'Kerucut':
 param1Label = 'Jari-jari Alas';
 param2Label = 'Tinggi';
 break;
 case 'Balok':
 param1Label = 'Panjang';
 param2Label = 'Lebar';
 param3Label = 'Tinggi';
 break;
 }
}

@override
Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text(widget.shape),
),
 body: SingleChildScrollView(
 padding: EdgeInsets.all(16.0),
 child: Form(
 key: _formKey,
 child: Column(
 children: [
 Text(
 'Masukkan Parameter untuk ${widget.shape}',
 style: TextStyle(fontSize: 20, fontWeight:
FontWeight.bold),
),
 SizedBox(height: 20),
 TextFormField(
 controller: param1Controller,
 decoration: InputDecoration(
 labelText: param1Label,
 border: OutlineInputBorder(),
),
 keyboardType: TextInputType.number,
),
 SizedBox(height: 20),
 if (param2Label.isNotEmpty)
 TextFormField(
 controller: param2Controller,
 decoration: InputDecoration(
 labelText: param2Label,
 border: OutlineInputBorder(),
),
),
],
),
),
),
);
}

```

```

),
 keyboardType: TextInputType.number,
),
 SizedBox(height: 20),
 if (param3Label.isNotEmpty)
 TextFormField(
 controller: param3Controller,
 decoration: InputDecoration(
 labelText: param3Label,
 border: OutlineInputBorder(),
),
 keyboardType: TextInputType.number,
),
 SizedBox(height: 20),
 ElevatedButton(
 onPressed: calculate,
 style: ElevatedButton.styleFrom(
 backgroundColor: Colors.green,
 padding: EdgeInsets.symmetric(vertical: 15.0,
horizontal: 30.0),
 shape: RoundedRectangleBorder(
 borderRadius: BorderRadius.circular(10),
),
),
 child: Text(
 'Hitung Luas & Keliling',
 style: TextStyle(fontSize: 18),
),
),
 SizedBox(height: 30),
 Text(
 'Luas: $resultArea',
 style: TextStyle(fontSize: 24, fontWeight:
FontWeight.bold, color: Colors.green),
),
 Text(
 'Keliling: $resultPerimeter',
 style: TextStyle(fontSize: 24, fontWeight:
FontWeight.bold, color: Colors.green),
),
],
),
),
),
);
}
}

```

## shape\_detail\_page.dart

```
// ignore_for_file: use_key_in_widget_constructors,
prefer_const_constructors_in_immutables,
library_private_types_in_public_api, prefer_const_constructors

import 'package:flutter/material.dart';

class ShapeDetailPage extends StatefulWidget {
 final String shape;

 ShapeDetailPage({required this.shape});

 @override
 _ShapeDetailPageState createState() => _ShapeDetailPageState();
}

class _ShapeDetailPageState extends State<ShapeDetailPage> {
 final _formKey = GlobalKey<FormState>();
 double resultArea = 0.0;
 double resultPerimeter = 0.0;

 TextEditingController param1Controller = TextEditingController();
 TextEditingController param2Controller = TextEditingController();
 TextEditingController param3Controller = TextEditingController();

 String param1Label = '';
 String param2Label = '';
 String param3Label = '';

 void calculate() {
 double param1 = double.tryParse(param1Controller.text) ?? 0.0;
 double param2 = double.tryParse(param2Controller.text) ?? 0.0;
 double param3 = double.tryParse(param3Controller.text) ?? 0.0;

 setState(() {
 switch (widget.shape) {
 case 'Kubus':
 resultArea = 6 * param1 * param1; // Luas Kubus
 resultPerimeter = 12 * param1; // Keliling Kubus
 break;
 case 'Bola':
 resultArea = 4 * 3.1416 * param1 * param1; // Luas Bola
 resultPerimeter = 2 * 3.1416 * param1; // Keliling Bola
 break;
 case 'Silinder':

```

```

 resultArea = 2 * 3.1416 * param1 * (param1 + param2); // Luas
Silinder
 resultPerimeter = 2 * 3.1416 * param1; // Keliling Silinder
 break;
 case 'Kerucut':
 resultArea = 3.1416 * param1 * (param1 + param2); // Luas
Kerucut
 resultPerimeter = 3.1416 * param1; // Keliling Kerucut
 break;
 case 'Balok':
 resultArea = 2 * (param1 * param2 + param2 * param3 + param3 *
param1); // Luas Balok
 resultPerimeter = 4 * (param1 + param2 + param3); // Keliling
Balok
 break;
 }
 });
}

@Override
void initState() {
 super.initState();
 setLabels();
}

void setLabels() {
 switch (widget.shape) {
 case 'Kubus':
 param1Label = 'Panjang Sisi';
 break;
 case 'Bola':
 param1Label = 'Jari-jari';
 break;
 case 'Silinder':
 param1Label = 'Jari-jari Alas';
 param2Label = 'Tinggi';
 break;
 case 'Kerucut':
 param1Label = 'Jari-jari Alas';
 param2Label = 'Tinggi';
 break;
 case 'Balok':
 param1Label = 'Panjang';
 param2Label = 'Lebar';
 param3Label = 'Tinggi';
 break;
 }
}

```

```

}

@override
Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text(widget.shape),
),
 body: SingleChildScrollView(
 padding: EdgeInsets.all(16.0),
 child: Form(
 key: _formKey,
 child: Column(
 children: [
 Text(
 'Masukkan Parameter untuk ${widget.shape}',
 style: TextStyle(fontSize: 20, fontWeight:
FontWeight.bold),
),
 SizedBox(height: 20),
 TextFormField(
 controller: param1Controller,
 decoration: InputDecoration(
 labelText: param1Label,
 border: OutlineInputBorder(),
),
 keyboardType: TextInputType.number,
),
 SizedBox(height: 20),
 if (param2Label.isNotEmpty)
 TextFormField(
 controller: param2Controller,
 decoration: InputDecoration(
 labelText: param2Label,
 border: OutlineInputBorder(),
),
 keyboardType: TextInputType.number,
),
 SizedBox(height: 20),
 if (param3Label.isNotEmpty)
 TextFormField(
 controller: param3Controller,
 decoration: InputDecoration(
 labelText: param3Label,
 border: OutlineInputBorder(),
),
 keyboardType: TextInputType.number,
),
],
),
),
),
);
}

```

```

),
 SizedBox(height: 20),
 // Perubahan font pada tombol 'Hitung Luas & Keliling'
dimulai di sini
 ElevatedButton(
 onPressed: calculate,
 style: ElevatedButton.styleFrom(
 backgroundColor: Colors.green,
 padding: EdgeInsets.symmetric(vertical: 15.0,
horizontal: 30.0),
 shape: RoundedRectangleBorder(
 borderRadius: BorderRadius.circular(10),
),
),
 child: Text(
 'Hitung Luas & Keliling',
 style: TextStyle(
 fontSize: 18,
 fontWeight: FontWeight.w600, // Mengatur ketebalan
font menjadi semi-bold
 fontFamily: 'Roboto', // Mengatur jenis font
(pastikan font ini tersedia)
 color: Colors.white, // Mengatur warna font
),
),
),
 // Perubahan font pada tombol 'Hitung Luas & Keliling'
selesai di sini
 SizedBox(height: 30),
 Text(
 'Luas: $resultArea',
 style: TextStyle(fontSize: 24, fontWeight:
FontWeight.bold, color: Colors.green),
),
 Text(
 'Keliling: $resultPerimeter',
 style: TextStyle(fontSize: 24, fontWeight:
FontWeight.bold, color: Colors.green),
),
],
),
),
),
);
}
}

```













